

# The Factory Model: How Coding Agents Changed Software Engineering

FEBRUARY 25, 2026

Something shifted recently with [agentic engineering](#) that “feels” like the level of abstraction changed again. Not the usual kind of shift where tools get marginally better and workflows gradually evolve. A step change. Developers who have been writing software for decades are describing it the same way: the center of gravity of the craft moved.

The most useful thing you can do right now is hold two ideas in tension simultaneously. **Coding has changed dramatically. Software engineering, at its core, has not.** That gap between the two is where the interesting story lives, and understanding it clearly is the difference between engineers who thrive in this era and engineers who get left behind by it.

I was reading Michael Truell (of Cursor’s) [thoughts](#) on this and wanted to expand on them.

## The arc of abstractions

**The history of software engineering is the history of raising abstraction.** We moved from bits to instructions, from instructions to functions, from functions to objects, from objects to services, from services to distributed systems. Every jump in the stack made individual developers more productive and expanded the total population of people who could participate in building software.

Assembly gave way to C. C gave way to managed languages and garbage collection. Managed languages gave way to frameworks, package ecosystems, and cloud

infrastructure. Each transition felt disruptive at the time. Each one, in hindsight, was simply the next step in a long and consistent arc.

**What we are living through right now is another step in that same arc. We are moving from writing code to orchestrating systems that write code.**

This framing was articulated by Grady Booch, who refers to it as [software's third age](#) — a new golden era defined by rising abstraction, where the job of the developer shifts from writing instructions to defining intent.

That framing matters because it tells you what to hold onto and what to let go of.

## Three generations of AI coding tools

It helps to be precise about the progression, because conflating the generations leads to underestimating how much has actually changed.

**The first generation was accelerated autocomplete.** Tools that predicted the next line, filled in boilerplate, and saved keystrokes on repetitive patterns. Useful. Genuinely time-saving. But the workflow stayed identical to what it had always been: you drove, the tool assisted. The feedback loop was: write code, run it, debug, repeat. The AI just reduced friction inside that loop.

**The second generation introduced synchronous agents.** You described a task in natural language. The model generated code. You reviewed it, corrected it, iterated toward a working result. This moved further up the stack. Less typing, more describing intent. But you were still present for every step. The agent was a collaborator, not an autonomous worker. You held the context, directed the next move, and caught mistakes in real time.

**The third generation introduced autonomous agents.** These agents can take a specification and run with it for thirty minutes, an hour, several hours and increasingly days. They set up environments, install dependencies, write tests, hit failures, research solutions online, fix the failures, write the implementation, test it again, set up services, and produce artifacts you can review. You hand them a task, move on to something else, and come back to logs, previews, and pull requests. You are no longer interacting line by line. You are defining outcomes and reviewing results. This is where [swarms of agents](#) and even [self-improving agents](#) come into play.

That changes the cadence of work in ways that are hard to fully communicate until you experience it. Tasks that were weekend projects three months ago are now something you kick off and check on thirty minutes later.

## The Factory Mental Model

**The most useful mental model for this new paradigm is that you are no longer just writing code. You are building the factory that builds your software.**

That factory consists of fleets of agents. Each agent has a task, a toolbelt (repositories, test runners, deployment scripts, documentation), context (specs, architecture decisions, prior constraints), and a feedback loop. Instead of hand-holding a single agent through a single task, you spin up many agents in parallel. One handles backend refactors. Another implements a feature. Another writes integration tests. Another updates documentation. You review outputs, give feedback, refine specs, and redeploy.

The analogy runs deeper than it might first appear. A factory has quality control. A factory has process documentation. A factory has inputs that need to be precisely specified or the outputs come out wrong. A factory stalls when the environment is unreliable. All of these properties map directly onto agentic software development, and taking the analogy seriously points you toward the investments that actually matter.

Inside teams that have adopted this model aggressively, a substantial portion of merged pull requests now originate from agents running autonomously in cloud environments. That is not theoretical anymore. It is production reality for a growing number of engineering organizations.

The [sentiments](#) from Cursor around “The developer’s job is becoming building the system that builds the software, the factory, not just the product” and “reviewing ideas is a lot more fun than reviewing code” ([video](#)) resonate with these points.

## There’s an onboarding parallel here

One of the most striking patterns in how agents actually behave is how closely their work loop mirrors onboarding a new engineer.

You hand them a spec. They break it into subtasks. They explore the codebase to understand the lay of the land. When they get stuck, they search commit history. They run git blame to figure out who last touched a subsystem. They escalate to the appropriate human for domain knowledge, via Slack or similar communication channels, and then continue. They iterate until the output meets the acceptance criteria.

That loop is familiar because it is how people work. The implication is significant. Slack and email are becoming interfaces between humans and agents, not just between humans and humans. Git history is evolving into a knowledge graph that agents navigate to understand architectural decisions. Documentation is becoming training material for autonomous execution.

If you want to think clearly about what investment to make in your codebase right now, ask yourself: could a new engineer, given only the documentation and commit history available, understand why the code is structured this way? If the answer is no, agents will struggle there too, and the leverage you could be getting will be limited.

## Your Spec is the Leverage

Here is the insight that reshapes how you think about your own value as an engineer.

If you can orchestrate twenty, thirty, fifty agents running in parallel, the difference between mediocre output and exceptional output comes down almost entirely to the quality of your specification. At that scale, vague thinking does not just slow you down. It multiplies. Ambiguous requirements propagate through dozens of parallel autonomous runs, each one going slightly wrong in a slightly different direction. Poor architectural decisions made upfront do not affect one implementation. They propagate across the entire fleet.

**You cannot write a spec that survives that environment unless you deeply understand the architecture, the integration boundaries, the edge cases, the failure modes, and the invariants that must never break.** The spec is not a prompt anymore. The spec is the product thinking made explicit.

This is why strong software engineers get more leverage from these tools than weak ones, not less. The mechanical work of typing code is being automated. The cognitive work of understanding systems is being amplified. Every hour you spend developing

genuine architectural understanding and systems thinking now pays dividends across an entire fleet of autonomous workers rather than just your own output.

## What hasn't really changed.

It is worth being precise here, because the hype around AI coding can create the impression that traditional software engineering skills have been deprecated.

They have not. Consider what agentic development still requires from you:

Clear requirements. **If you cannot articulate what success looks like in a way that can be evaluated, no amount of autonomous execution will produce it. Agents cannot clarify requirements they are never given.** They will fill the gaps with assumptions, and those assumptions compound.

Strong abstractions. An agent given a well-designed system with clear module boundaries, coherent interfaces, and good separation of concerns will produce better results than an agent given a tangled codebase where everything depends on everything else. Clean architecture does not become less valuable when agents are doing the implementation. It becomes more valuable, because agents amplify the properties of the system they are working in.

Reliable tests. This deserves its own section.

Careful tradeoffs. Agents optimize for the stated objective. They do not naturally balance competing concerns, anticipate second-order effects, or flag when a technically correct solution is the wrong product decision. That judgment still lives with you.

**Human oversight.** Agents do impressive work. They also make confident mistakes. The output quality is high enough to get past casual review, which means the bar for your review skills actually increases, not decreases.

## Why tests matter more than ever

Good tests and Test-driven development (TDD) were already good practice. In an agentic workflow, they become something close to mandatory.

The idea is precise enough to be worth stating clearly. Red/green TDD means you write the tests before you write the implementation. You confirm the tests fail (red phase). Then you iterate on the implementation until the tests pass (green phase). That sequence is not optional ceremony. It is the mechanism that gives you confidence the implementation is actually doing what you think it is.

With a single developer writing code, the downside of skipping test-first development is that you might write a test that passes regardless of whether your implementation is correct, or miss edge cases that get caught later as regressions. Those are real costs. They are manageable.

With a fleet of agents generating code across dozens of parallel tasks, the costs compound severely. **An agent optimizing for passing tests will find ways to pass them. If the tests were written after the implementation, they are likely testing what the implementation happens to do rather than what it should do.** You now have a large surface area of code with a test suite that confirms the wrong thing. A comprehensive, test-first suite is by far the most effective lever you have for ensuring that autonomous output is actually correct and for protecting existing functionality as the codebase grows.

“Red/green TDD” is a shorthand every good model understands. It captures a specific discipline: write tests first, confirm they fail before implementing, make them pass through correct implementation rather than by gaming the test. Telling an agent to use red/green TDD is one of the highest-leverage instructions you can give at the start of a task.

## **The unsolved problem is verification, not generation.**

Generation is not the bottleneck anymore. Verification is.

Agents can produce impressive output. The challenge is knowing with confidence whether that output is correct. Several factors make this harder than it first appears.

Tests that pass before a change does not mean they will catch regressions introduced by the change. Agents can write tests that are technically valid but miss the cases that matter. UI verification remains brittle, with visual and behavioral regressions slipping through because automated tools are not yet reliable enough to catch them all.

**Context window** limitations mean that agents working on large codebases may miss

important constraints or patterns that exist outside the window they are currently reasoning over. Flaky environments, which a single developer encounters as an annoying edge case and works around, become systemic blockers when you have forty agents hitting the same flaky test simultaneously. The factory stalls.

The infrastructure that needs to exist to support this model at scale includes better automated regression detection, artifact-level validation that goes beyond diffing changed lines, reliable and fast environment provisioning, and **guardrails** that hold up under parallel workloads. These are active areas of investment. They are not solved.

Until verification catches up with generation, human review is not optional overhead. It is the safety system. The appropriate response to impressive agent output is not to trust it because it looks good. It is to have the architectural understanding and testing discipline to evaluate it rigorously.

## The new shape of high-leverage engineering

The engineers who will have the most impact in this era will not be distinguished by how fast they type or how well they remember syntax. They will be distinguished by a different set of capabilities.

Systems thinking. The ability to hold a complex architecture in mind, understand how components interact, and anticipate how a change in one place affects behavior elsewhere. This is harder to develop than typing speed and far more valuable when you are managing a fleet of agents whose outputs you have to integrate.

Problem decomposition. Knowing how to break a large, ambiguous goal into well-scoped subtasks that an agent can execute reliably. Tasks that are too large tend to go off-track. Tasks that are poorly scoped get interpreted incorrectly. The skill of decomposing problems well, and then verifying that the decomposition was right, is a genuine craft.

Architectural judgment. Understanding why a system is designed the way it is, what properties it is optimizing for, and what tradeoffs were made. Agents can implement. They cannot judge whether what they are implementing is the right design.

Specification clarity. The ability to write requirements that are unambiguous, complete with respect to the important edge cases, and structured in a way that makes

evaluation straightforward. Vague specs produce vague results. Precise specs multiply into precise implementations.

Output evaluation. The taste to recognize when something looks correct but is not, when an implementation solves the stated problem but creates a new one, when the architecture of the solution does not fit the architecture of the rest of the system. This judgment is not automatable.

Orchestration skill. The practical ability to manage multiple parallel workstreams, give effective feedback on agent outputs, recognize when an agent needs to be redirected versus retasked, and maintain coherence across a fleet of autonomous workers.

None of these are new skills, exactly. Good engineers have always needed them. What has changed is their relative importance. The mechanical parts of software development are being increasingly handled by machines. The cognitive parts are being amplified.

## What's the bigger picture?

New website creation is up 40% year over year. New iOS apps are up nearly 50%. GitHub code pushes jumped 35% in the US. All of these metrics were flat for years before late 2024. The [graphs](#) look like hockey sticks. People who have never written a line of code are building and launching software.

Keep in mind, we can and should note that more quantity does not necessarily mean better quality. But the fact remains that the barrier to creating software has dropped dramatically, and that is a fundamental shift in the landscape of software engineering.

**The barrier to creating software has genuinely dropped. That is not hype. What it means for professional engineers is not that their skills are less valuable, but that the skills that matter have shifted up the stack, as they have in every previous transition.**

The developers who thrived after the move from assembly to C were not the ones who could write the most clever assembly. They were the ones who understood what the machine needed to do and could express that intent clearly in a higher-level language. The developers who thrived after the move to managed languages and frameworks were not the ones most resistant to garbage collection. They were the ones who saw the freed-up cognitive capacity as an opportunity to solve harder problems.

The developers who will thrive in the agentic era are the ones who understand this as another step in the same arc and invest accordingly. Not in resisting the tools. Not in deferring to them uncritically. In developing the judgment, clarity, and systems thinking that make the tools maximally effective.

That means writing better specs. Investing in test infrastructure. Developing genuine architectural understanding rather than surface familiarity. Building the taste to evaluate output rigorously. Practicing problem decomposition until it becomes second nature.

The era of programming as primarily a keystroke activity is over. The era of programming as primarily a thinking and judgment activity has been accelerating for decades and just shifted into a higher gear.

The factory model is not a metaphor about losing control of software. It is a metaphor about building leverage. The engineers who understand that will build the most interesting things of the next decade.

---

## Related reading

|                                                |          |
|------------------------------------------------|----------|
| <a href="#">Brownfield Agentic Engineering</a> | Sep 2026 |
| <a href="#">Agentic Skill Decay</a>            | Aug 2026 |
| <a href="#">Audit your Agent files</a>         | Aug 2026 |

[Browse all topic guides →](#)

### FOUND THIS USEFUL?

#### Go deeper in *Leading Effective Engineering Teams*

My O'Reilly book on growing as an engineer, tech lead, or manager: hiring, delegation, communication,

and leading teams that ship.

Read the book



Addy Osmani is an engineering and evangelism leader and a Member of Technical Staff at Anthropic, where he works on Claude Code. He spent over 14 years at Google leading developer experience across Chrome and, in recent years, AI (Gemini, coding agents, and agentic engineering), most recently as a Director at Google Cloud AI.

[Tweet](#)

[Bluesky](#)

[Mastodon](#)

[Threads](#)

[LinkedIn](#)

[Share](#)

Want more? **Subscribe to my free newsletter:**

example@gmail.com

Subscribe

**Disclaimer:** The views and opinions expressed on this site are my own.

© Copyright 2026 Addy Osmani